

PIN-0005

P-PAS+ Adaptive Scheduling With Persistent Stillpoint Restore

Independent port, feature-equivalence validation, and mixed-pressure evaluation

Stillpoint Lab 4rce.com Digital Technologies GmbH

Published 4 September 2026

Revised 4 September 2026

Version 1.1

Status Validated active profile

CANONICAL PUBLICATION

HTML: <https://stillpointlab.dev/pin-0005-ppas-plus.html>

PDF: <https://stillpointlab.dev/pdf/pin-0005-ppas-plus.pdf>

Abstract

Long cold prefills can interfere with active decoding in long-context large-language-model serving. We independently ported P-PAS+ prefill-pressure adaptive scheduling to a production-like Qwen3.8-27B profile running vLLM 0.28.0 while retaining persistent LMCACHE restore, MTP-3 speculative decoding, FlashInfer XQA, FP8 KV cache, and a 131,072-token context window. The candidate passed all declared correctness, compatibility, persistence, and recovery gates and completed 40 of 40 repeated workload executions. In a preliminary contemporaneous paired canary with two repetitions per scenario, P-PAS+ increased median decode throughput by 4.90% and reduced median decode wall time by 4.52% when one 100k-token cold prefill overlapped three active decodes. It was effectively neutral in the two-prefill/two-decode case. A later control failure was isolated to insufficient GPU activation headroom rather than scheduler correctness; that result is reported as a failure and not used to claim universal memory benefit. The evidence supports P-PAS+ as a validated, switchable profile for the declared stack, not as a universal vLLM speed claim.

Keywords—P-PAS+, vLLM, long-context inference, LMCACHE, Qwen3.8, MTP, prefill-decode interference

Document notice. This PDF is an independently typeset archival research paper. The canonical mutable publication is <https://stillpointlab.dev/pin-0005-ppas-plus.html>. Quantitative claims are limited to the declared system, workloads, and repetitions.

Contents

1. Executive summary	3
2. Problem statement	3
3. Research questions and falsification criteria	3
4. Related work and attribution	3
5. System under test	4
6. Stillpoint adaptation	4
7. Test suite and experimental method	5
8. Results	5
8.1. Preliminary paired canary	5
8.2. Clean candidate validation	5
9. Failure analysis and corrective experiment	6
10. Discussion	6
11. Threats to validity and non-claims	6
12. Security, privacy, and disclosure	6
13. Reproducibility and artifacts	7
14. Conclusion	7
15. Acknowledgements	7
16. References	7
17. Revision and run history	7

1. Executive summary

Problem	Large cold prefills compete with active decoding for the scheduler token budget.
Contribution	A vLLM 0.28 P-PAS+ port preserving LMCache, MTP-3, long context, APIs, tools, reasoning, and image processing.
Strongest result	In the target paired scenario, median decode throughput rose 4.90% and median decode wall time fell 4.52% (two repetitions per arm).
Validation	All declared candidate gates passed; 40/40 repeated candidate workloads completed without request error.
Current status	Validated, switchable active profile. Promotion claims remain bounded by the incomplete extended paired campaign.
Key limitation	The strongest gain is workload-specific and based on two paired repetitions; it is not a universal speed or memory claim.

2. Problem statement

Long-context inference combines two latency-sensitive phases. Prefill processes the input context and is compute intensive; decode emits tokens iteratively and is sensitive to interruption. A large cold or partially cached prefill can consume most of a scheduler iteration’s token budget while interactive requests are decoding. Persistent Stillpoint restore avoids prefill work for exact cached prefixes, but cannot remove every cold prefill. Adaptive scheduling is therefore complementary: restore removes avoidable work, while P-PAS+ limits interference from unavoidable work.

3. Research questions and falsification criteria

- **RQ1.** Can P-PAS+ be ported to vLLM 0.28 and Qwen3.8 without violating the LMCache hybrid-alignment invariant?
- **RQ2.** Does the candidate preserve deterministic output, long-context retrieval, restart restore, MTP-3, API compatibility, tool calling, reasoning parsing, and image processing?
- **RQ3.** Does pressure capping improve active decode behavior when one very large prefill overlaps several decodes?
- **RQ4.** Does it avoid material regression when two medium-long prefills overlap two decodes?

H1 is falsified by any correctness, cache-integrity, compatibility, engine-health, or rollback failure. **H2** expects lower decode wall time and higher decode throughput in the one-large-prefill scenario. **H3** expects the dual-prefill case to remain within ordinary run variance. The experiment is stopped on corruption, NaN output, worker death, timeout, or failed recovery.

4. Related work and attribution

P-PAS and the P-PAS+ heterogeneous-pressure extension were developed by Timo Sämann and published with an open implementation [1, 2]. We thank the author for making the work available for independent evaluation.

Stillpoint’s contribution is an adaptation and validation on a different model, vLLM release, GPU class, and persistent-cache architecture. It is not the original P-PAS+ contribution.

The scheduling work addresses a layer complementary to persistent hybrid-state restore. Exact-prefix restore avoids computation when a prior standing place exists; P-PAS+ governs remaining scheduler contention when a prefill cannot be avoided.

5. System under test

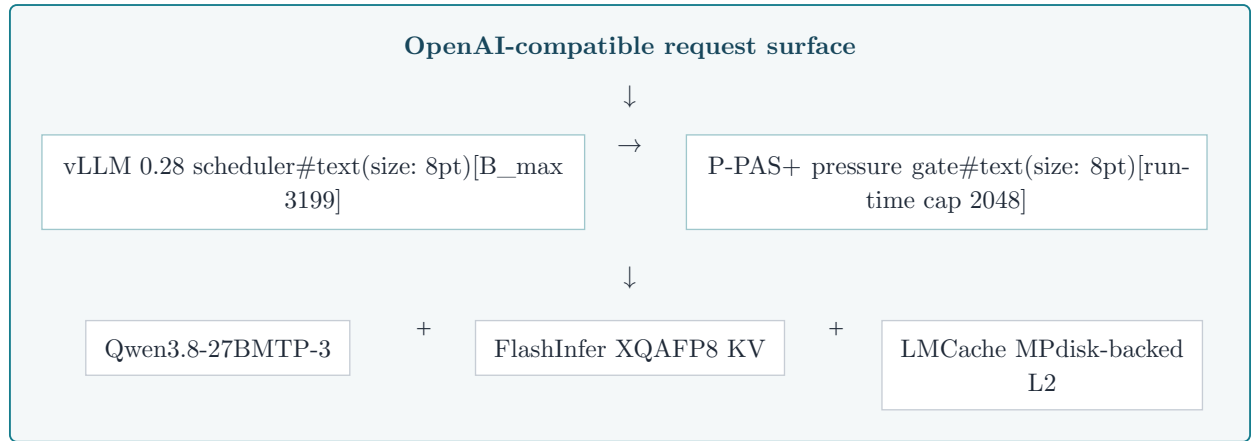


Figure 1: Figure 1. Integration boundary. P-PAS+ changes only the per-iteration scheduled-token cap under pressure; the complete serving and persistence stack remains enabled.

Model	Qwen3.8-27B W4A16 AWQ
Runtime	vLLM 0.28.0; tensor parallelism 2
Accelerators	2 × NVIDIA RTX PRO 2000 Blackwell, 16 GiB each, SM120
Attention	FlashInfer XQA; PIECEWISE CUDA graphs
KV cache	FP8; validated hybrid geometry
Speculative decode	MTP-3
Persistent cache	LMCache multiprocess connector; disk-backed L2
Context	131,072 tokens
Hybrid invariant	Mamba align; unified block size $N = 1600$

6. Stillpoint adaptation

Upstream scheduler constants cannot be copied directly. The tested hybrid stack requires configured `max_num_batched_tokens` in $[1600, 3200)$. Setting the static budget to exactly 1600 serializes useful work, while exceeding the interval violates the validated LMCACHE input-budget geometry. We retain the configured maximum and apply only a runtime cap:

$$B_{\max} = 3199, \quad B_{\text{cap}} = 2048, \quad T_{\text{large}} = 65536.$$

Pressure rule A activates when at least two prefills and one decode are active. Rule B activates when exactly one running prefill has at least 65,536 tokens remaining and at least two decodes are active. The configured maximum remains 3199; therefore MTP draft-slot accounting and LMCACHE hybrid validation retain the established behavior.

7. Test suite and experimental method

The candidate was implemented in a full, isolated runtime profile rather than through `PYTHONPATH` overlay or in-place modification of the control environment. Profile switching was mutually exclusive and health checked. Copied artifacts and imported evidence were verified with SHA-256.

The acceptance suite covered:

- deterministic generation and repeated 200-token outputs;
- buried-key retrieval at approximately 13k, 30k, 60k, 66.7k, 112k, and 130k tokens;
- LMCache store and restart restore from 5k through 112k;
- model discovery, non-streaming completion, and SSE streaming;
- German output, reasoning parsing, native tool calling, and deterministic coding;
- real image processing and eight-request queue submission;
- short, agent-shaped, 111k, concurrent, heterogeneous, and mixed-pressure workloads.

For the preliminary paired canary, the same host, prompt shapes, output lengths, cache controls, and trace order were used for both profiles. Each pressure scenario was repeated twice. Reported values are medians. The later clean candidate suite used five repetitions per workload. These counts are insufficient for broad population-level inference and are presented as measured engineering evidence.

8. Results

8.1. Preliminary paired canary

Scenario and metric	Control	P-PAS+	Change
1 × 100k prefill + 3 decodes: decode wall	456.19 s	435.59 s	−4.52%
Same: decode throughput	1.751 tok/s	1.837 tok/s	+4.90%
Same: makespan	457.07 s	435.88 s	−4.64%
2 × 56k prefills + 2 decodes: decode wall	134.13 s	134.44 s	+0.23%
Same: decode throughput	5.957 tok/s	5.944 tok/s	−0.23%
Same: makespan	134.12 s	134.65 s	+0.40%

The target one-large-prefill case favored P-PAS+ on all three declared measures. The dual-prefill case differed by no more than 0.40% and is interpreted as effectively neutral, not as evidence of improvement.

8.2. Clean candidate validation

Workload	Median wall	Median rate	Runs
Short decode, 512 output	9.522 s	54.491 tok/s	5/5
8k agent, 512 output	17.698 s	49.501 tok/s	5/5
111k prompt, 256 output	142.437 s	43.521 tok/s	5/5
4 concurrent short	12.519 s	163.597 aggregate tok/s	5/5
4 concurrent 111k	569.211 s	0.899 aggregate tok/s	5/5
Heterogeneous 16k–130k	304.698 s	2.310 aggregate tok/s	5/5
1 large prefill + 3 decodes	474.043 s	1.730 decode tok/s	5/5
2 prefills + 2 decodes	140.252 s	5.698 decode tok/s	5/5

All 40 candidate executions completed without request error. MTP acceptance was 59.15% in the steady suite and 61.75% under mixed pressure.

9. Failure analysis and corrective experiment

The extended control arm started successfully, passed health checks, and completed five short-decode repetitions. During the 8k-agent sequence both tensor-parallel workers exhausted CUDA memory in the MTP proposal path while attempting a 54 MiB allocation. Only approximately 35.5 MiB remained free on each 15.48 GiB device. The API subsequently surfaced `EngineDeadError` and HTTP 500.

A focused unchanged retry reproduced the same failure, establishing that it was not a transient client error. The configuration reserved 93% of GPU memory for vLLM while LMCache and compiled activation buffers remained active. Reducing `gpu_memory_utilization` symmetrically from 0.93 to 0.91 created additional activation headroom without disabling LMCache, MTP-3, FP8 KV, FlashInfer, or either scheduler. Under the corrected envelope, the control completed five of five focused 8k/512 runs. This corrective observation isolates a configuration headroom defect; it does not prove that P-PAS+ intrinsically prevents OOM.

10. Discussion

The results answer RQ1 and RQ2 affirmatively for the declared stack: the port retained the hybrid budget invariant and passed the complete candidate gate set. RQ3 receives preliminary support because all target-case medians favored P-PAS+. RQ4 is also consistent with the hypothesis: the non-target dual-prefill case remained nearly unchanged.

The mechanism is intentionally narrow. P-PAS+ does not accelerate model kernels or eliminate prefill computation. It changes the distribution of scheduled tokens during a defined pressure state, protecting decode progress at the cost of constraining prefill advancement. Benefits therefore depend on workload overlap and may disappear outside that regime.

11. Threats to validity and non-claims

- The paired canary used two repetitions per scenario; it does not establish tail distributions, confidence intervals, or fleet-wide goodput.
- The strongest effect is tied to one workload shape on one dual-GPU host.
- The dual-prefill scenario was neutral and slightly favored the control numerically.
- The control OOM was a shared memory-envelope problem and is not evidence that P-PAS+ reduces memory use.
- Results do not establish portability to other models, vLLM versions, accelerators, cache geometries, or scheduler policies.
- Historical champion values provide context only and do not replace a complete contemporaneous corrected campaign.

12. Security, privacy, and disclosure

No credentials, private endpoints, personal data, or internal addressing are included. Published setup details are limited to factors required to interpret the experiment. Raw artifacts remain in the private engineering evidence repository pending disclosure review. Third-party code and research retain their original ownership and licenses.

13. Reproducibility and artifacts

The engineering record retains the exact scheduler diff, isolated profile configuration, raw JSON outputs, service journals, state captures, and SHA-256 manifests. The upstream implementation is pinned to commit 50340d20a2da5ab6fec704dceb83cecf8734dbb5. Public artifact links will be added only after disclosure review.

14. Conclusion

P-PAS+ is a validated switchable profile for the declared Stillpoint stack. It preserves the full tested capability surface and completed 40/40 repeated candidate workloads. Preliminary paired evidence supports a workload-specific decode-protection benefit when one very large cold prefill overlaps active decoding, while the dual-prefill case remained neutral. The evidence does not justify a universal speed or memory claim. A complete corrected paired campaign remains the appropriate promotion gate.

15. Acknowledgements

We thank Timo Sämann for publishing P-PAS/P-PAS+ and its implementation openly. We also acknowledge the vLLM, LMCache, and FlashInfer communities whose software forms the evaluated stack.

16. References

1. T. Sämann, “P-PAS: Prefill-Pressure Adaptive Scheduling for Long-Context LLM Serving,” arXiv:2608.15171, 2026. <https://arxiv.org/abs/2608.15171>.
2. T. Sämann, “ppas-vllm,” ppas-plus branch, commit 50340d20a2da5ab6fec704dceb83cecf8734dbb5, Apache-2.0, accessed 4 September 2026. <https://github.com/TimoSaemann/ppas-vllm/tree/ppas-plus>.
3. vLLM Project, “vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention,” software version 0.28.0, accessed 4 September 2026. <https://github.com/vllm-project/vllm>.
4. LMCache Project, “LMCache,” persistent KV-cache software, accessed 4 September 2026. <https://github.com/LMCache/LMCache>.
5. FlashInfer Project, “FlashInfer,” attention kernels for LLM serving, accessed 4 September 2026. <https://github.com/flashinfer-ai/flashinfer>.

17. Revision and run history

- **2026-09-04, version 1.0:** preliminary paired canary and clean candidate validation published.
- **2026-09-04, version 1.1:** independently typeset paper; focused control OOM reproduction and symmetric activation-headroom correction documented.

Legal and publication notice

© 2026 4rce.com Digital Technologies GmbH. All rights reserved. Stillpoint and Stillpoint Lab are research project names of 4rce.com. Original Stillpoint text, diagrams, benchmark analyses, and publication design are owned by 4rce.com Digital Technologies GmbH unless expressly stated otherwise. Third-party papers, software, code, product names, and trademarks remain the property of their respective owners. Citation and acknowledgement of upstream work do not transfer ownership.

Imprint and privacy: <https://stillpointlab.dev/imprint.html>